



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования

**«Дальневосточный федеральный университет»
(ДВФУ)**

**ИНСТИТУТ МАТЕМАТИКИ И КОМПЬЮТЕРНЫХ
ТЕХНОЛОГИЙ**

Лабораторная работа 3

Дисциплина «Теория вероятностей и математическая статистика»

Студент группы Б9123-01.03.02ии

Есакова Елизавета Михайловна

г. Владивосток

2025

ЛР №3 | Эмпирическая функция распределения
Целью данной лабораторной работы является изучение понятия эмпирической функции распределения.

<https://colab.research.google.com/drive/1hCs649rEvBp5JhqjhcKy5mwr88NvvuiG#scrollTo=TE-2ME2f3Cti>

Эмпирическая функция распределения:

Эмпирической функцией распределения (функцией распределения выборки) называется функция $F^*(x)$, которая определяется для каждого значения x относительную частоту события $X < x$:

$$F^*(x) = \frac{n_x}{n}$$

n_x – число выборочных значений, меньше x
 n – объем выборки

Доверительный интервал:

Определение

Доверительным интервалом параметра θ с уровнем доверия γ называется интервал (a_n, b_n) такой, что

$$P(a_n < \theta < b_n) \geq \gamma,$$

причём $a_n = a_n(X_1, X_2, \dots, X_n)$, $b_n = b_n(X_1, X_2, \dots, X_n)$.

Доверительный интервал для функции распределения

Имеется выборка $X_1, X_2, \dots, X_n$ из X .

$(L(x), U(x))$ – доверительный интервал для $F_X(x)$ с уровнем доверия $1 - \alpha$, т.е.

$$P(U(x) < F_X(x) < L(x)) \geq 1 - \alpha,$$

где

$$\begin{aligned} L(x) &= \max \left(\hat{F}_n(x) - \varepsilon_n; 0 \right), \\ U(x) &= \min \left(\hat{F}_n(x) + \varepsilon_n; 1 \right), \\ \varepsilon_n &= \sqrt{\frac{1}{2n} \ln \left(\frac{2}{\alpha} \right)}. \end{aligned}$$



Собственная реализация в питоне:

```
def empiricheskaya_fnc (sample, x):  
    n = len(sample)
```

```

alpha = 0.05
empirich= np.mean(sample <= x)

epsilon = np.sqrt(np.log(2 / alpha) / (2 * n))

L_x = max(0, empirich - epsilon)
U_x = min(1, empirich + epsilon)

return empirich, L_x, U_x

```

$\alpha = 0.05$ – значение для границы 95%-го доверительного интервала
`empirich= np.mean(sample <= x)` – поиск эмпирической функции
`L_x` – нижняя граница доверительного интервала
`U_x` – верхняя граница доверительного интервала

Реализация с помощью `scipy.stats` (так как выборки большие ($n > 30$), использую `norm.interval`, а не `t.interval`):

```

import numpy as np
from scipy.stats import ecdf, norm

'''co scipy.stats'''

def empiricheskaya_fnc_scipy(sample, x):

    empirich = ecdf(x)

    n = len(sample)
    alpha = 0.05

    stand_err = np.sqrt(empirich * (1 - empirich) / n)

    ci = norm.interval(1 - alpha, loc=empirich, scale=stand_err)

    L_x = np.maximum(0, ci[0])
    U_x = np.minimum(1, ci[1])

    return empirich, L_x, U_x

```

$\alpha = 0.05$ – значение для границы 95% доверительного интервала
`empirich = ecdf(x)` – поиск эмпирической функции
`stand_err = np.sqrt(empirich * (1 - empirich) / n)` – поиск стандартной ошибки
`norm.interval` – поиск доверительного интервала где:

- $1 - \alpha$ - уровень доверия (0.95 для 95% доверительного интервала).
- `loc=empirich` – среднее значение (значение `ecdf`).
- `scale=stand_err` - стандартная ошибка

L_x – нижняя граница доверительного интервала
 U_x – верхняя граница доверительного интервала

Код для построения графиков распределений со своей эмпирической функцией:

```
import scipy.stats as stats
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(4)

uniform_params = {'loc': 5, 'scale': 10}
bernoulli_params = {'p': 0.7}
binomial_params = {'n': 20, 'p': 0.4}
normal_params = {'loc': 5, 'scale': 2}

samples = {
    'Uniform_100': stats.uniform.rvs(size=100, **uniform_params),
    'Uniform_1000': stats.uniform.rvs(size=1000, **uniform_params),
    'Bernoulli_100': stats.bernoulli.rvs(size=100, **bernoulli_params),
    'Bernoulli_1000': stats.bernoulli.rvs(size=1000, **bernoulli_params),
    'Binomial_100': stats.binom.rvs(size=100, **binomial_params),
    'Binomial_1000': stats.binom.rvs(size=1000, **binomial_params),
    'Normal_100': stats.norm.rvs(size=100, **normal_params),
    'Normal_1000': stats.norm.rvs(size=1000, **normal_params)
}

def empiricheskaya_fnc(sample, x):
    n = len(sample)
    alpha = 0.05
    empirich = np.mean(sample <= x)
    epsilon = np.sqrt(np.log(2 / alpha) / (2 * n))
    L_x = max(0, empirich - epsilon)
    U_x = min(1, empirich + epsilon)
    return empirich, L_x, U_x

colors = ['blue', 'green', 'red']

for name, sample in samples.items():
    if 'Uniform' in name:
        x_range = np.linspace(uniform_params['loc'], uniform_params['loc']
+ uniform_params['scale'], 1000)
        true_cdf = stats.uniform.cdf(x_range, **uniform_params)
        plot_type = 'plot'
```

```

elif 'Bernoulli' in name:
    x_range = [0, 1]
    true_cdf = [1 - bernoulli_params['p'], 1]
    plot_type = 'step'
elif 'Binomial' in name:
    x_range = np.arange(0, binomial_params['n'] + 1)
    true_cdf = stats.binom.cdf(x_range, **binomial_params)
    plot_type = 'step'
elif 'Normal' in name:
    x_range = np.linspace(normal_params['loc'] - 4 *
normal_params['scale'],
                        normal_params['loc'] + 4 *
normal_params['scale'], 1000)
    true_cdf = stats.norm.cdf(x_range, **normal_params)
    plot_type = 'plot'

empirical_cdf = []
lower_bounds = []
upper_bounds = []
for x in x_range:
    emp, L_x, U_x = empiricheskaya_fnc(sample, x)
    empirical_cdf.append(emp)
    lower_bounds.append(L_x)
    upper_bounds.append(U_x)

plt.figure(figsize=(10, 6))

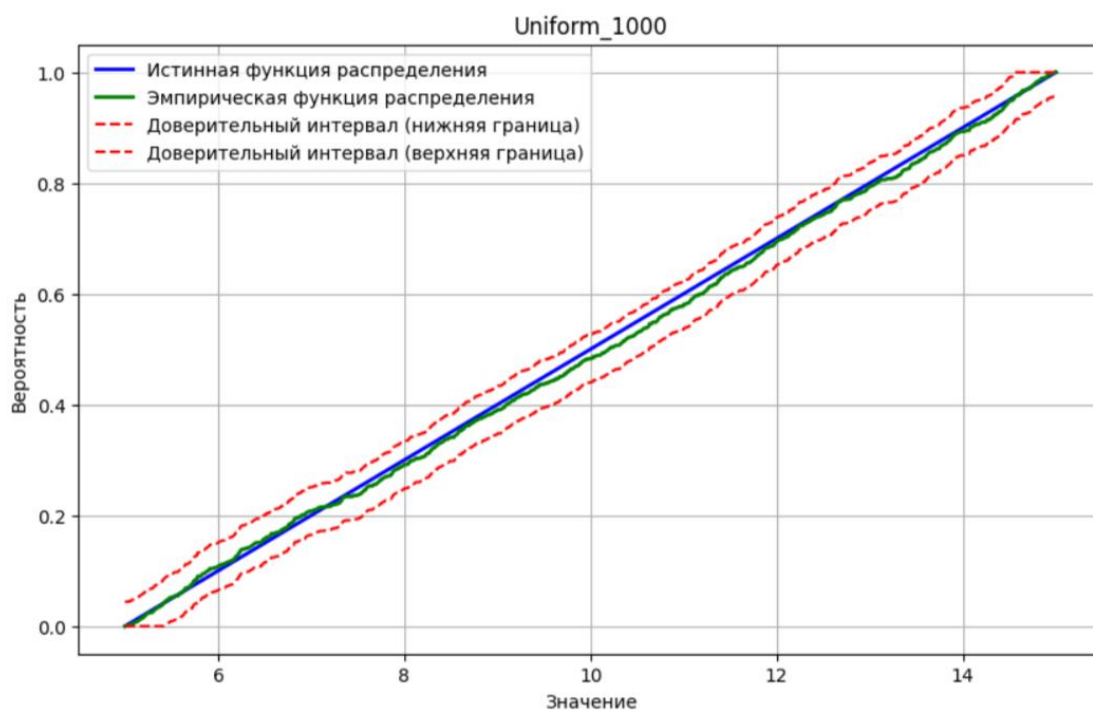
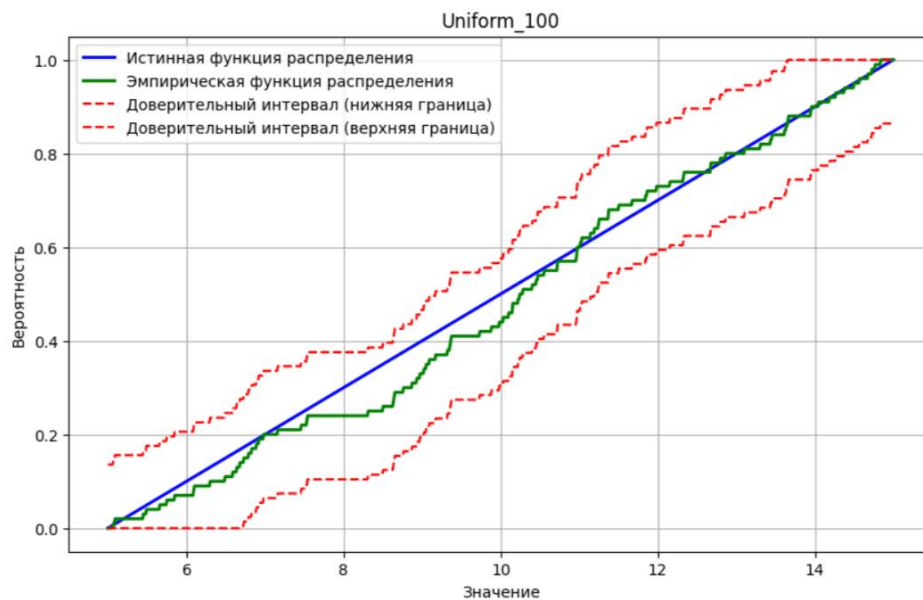
if plot_type == 'plot':
    plt.plot(x_range, true_cdf, color=colors[0], label='Истинная
функция распределения', lw=2)
    plt.plot(x_range, empirical_cdf, color=colors[1],
label='Эмпирическая функция распределения', lw=2)
    plt.plot(x_range, lower_bounds, color=colors[2], linestyle='--',
label='Доверительный интервал (нижняя граница)')
    plt.plot(x_range, upper_bounds, color=colors[2], linestyle='--',
label='Доверительный интервал (верхняя граница)')
elif plot_type == 'step':
    plt.step(x_range, true_cdf, where='post', color=colors[0],
label='Истинная функция распределения', lw=2)
    plt.step(x_range, empirical_cdf, where='post', color=colors[1],
label='Эмпирическая функция распределения', lw=2)
    plt.step(x_range, lower_bounds, where='post', color=colors[2],
linestyle='--', label='Доверительный интервал (нижняя граница)')
    plt.step(x_range, upper_bounds, where='post', color=colors[2],
linestyle='--', label='Доверительный интервал (верхняя граница)')

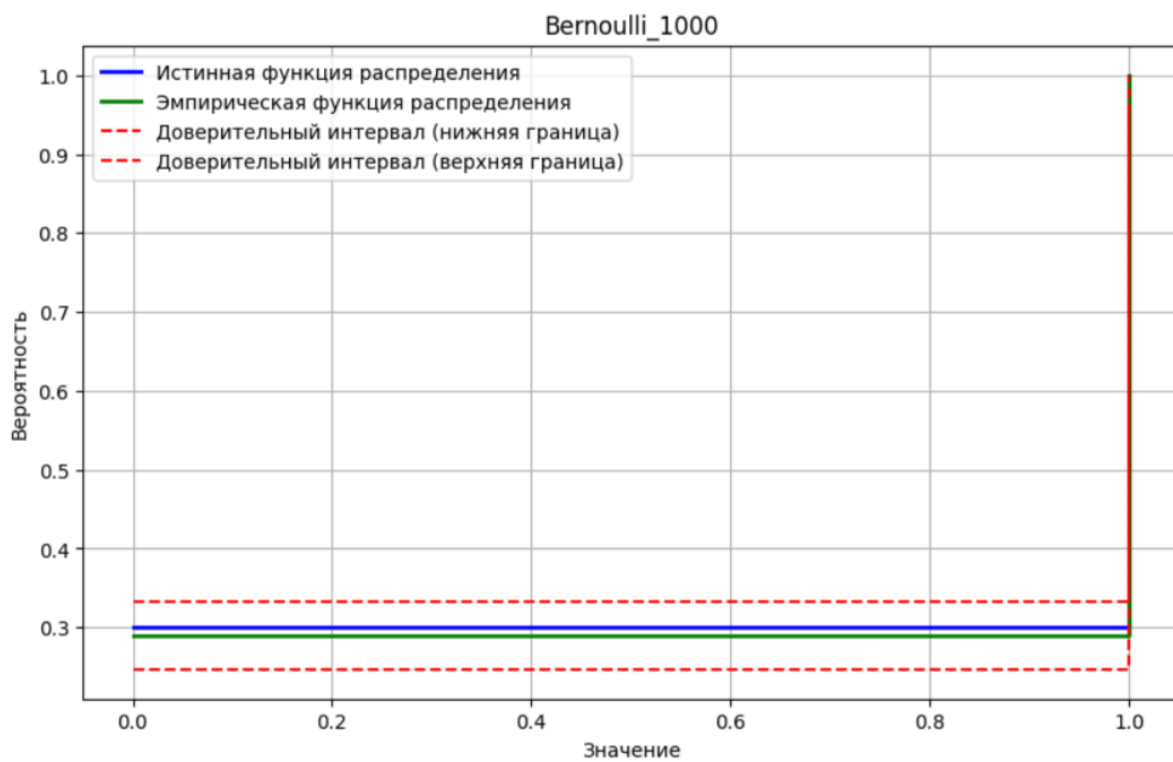
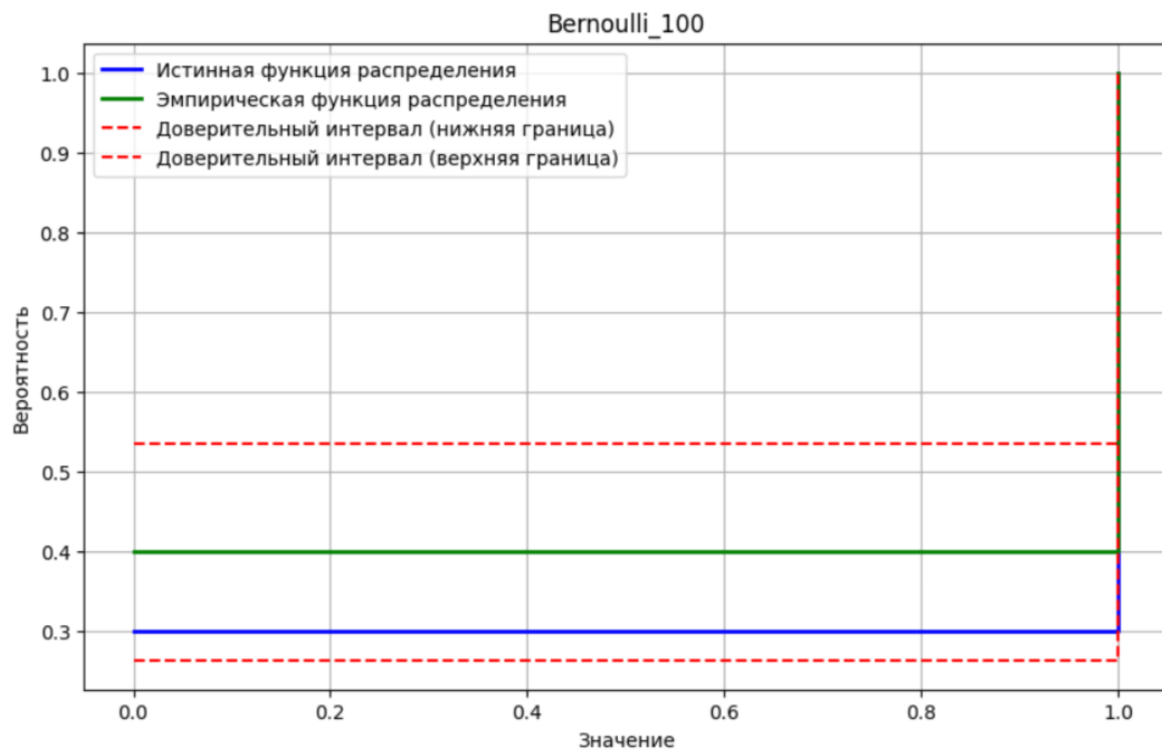
plt.title(f'{name}')
plt.xlabel('Значение')
plt.ylabel('Вероятность')

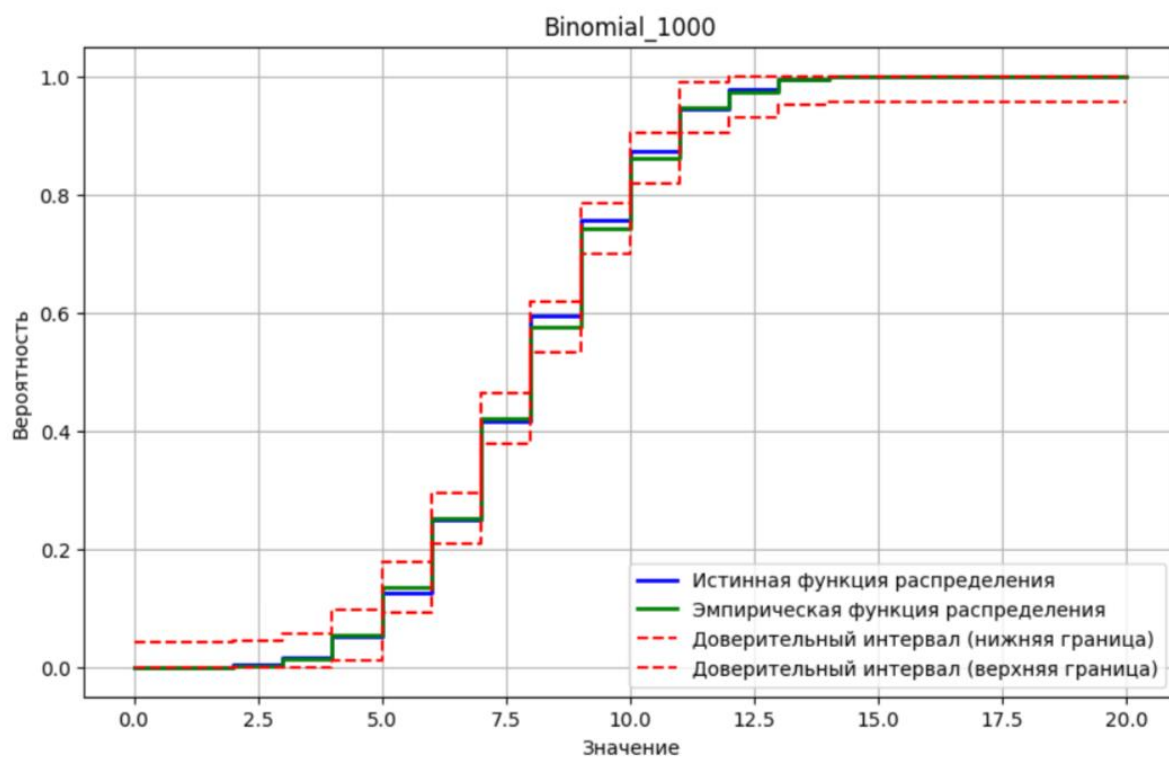
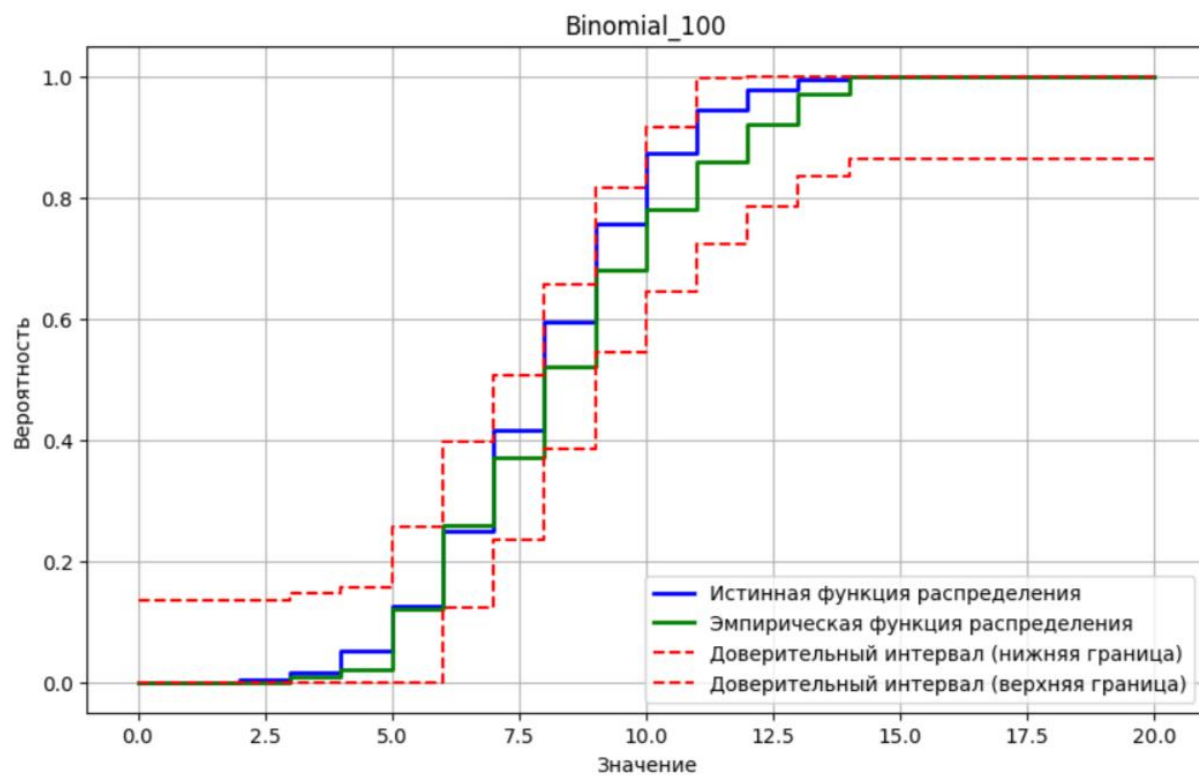
```

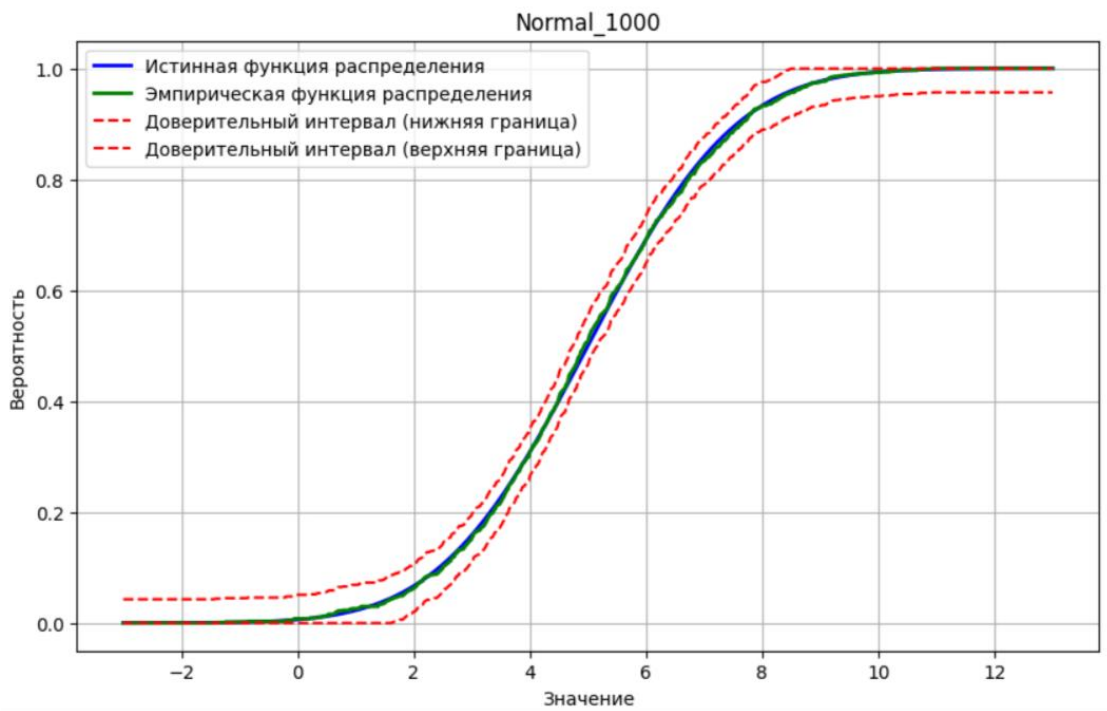
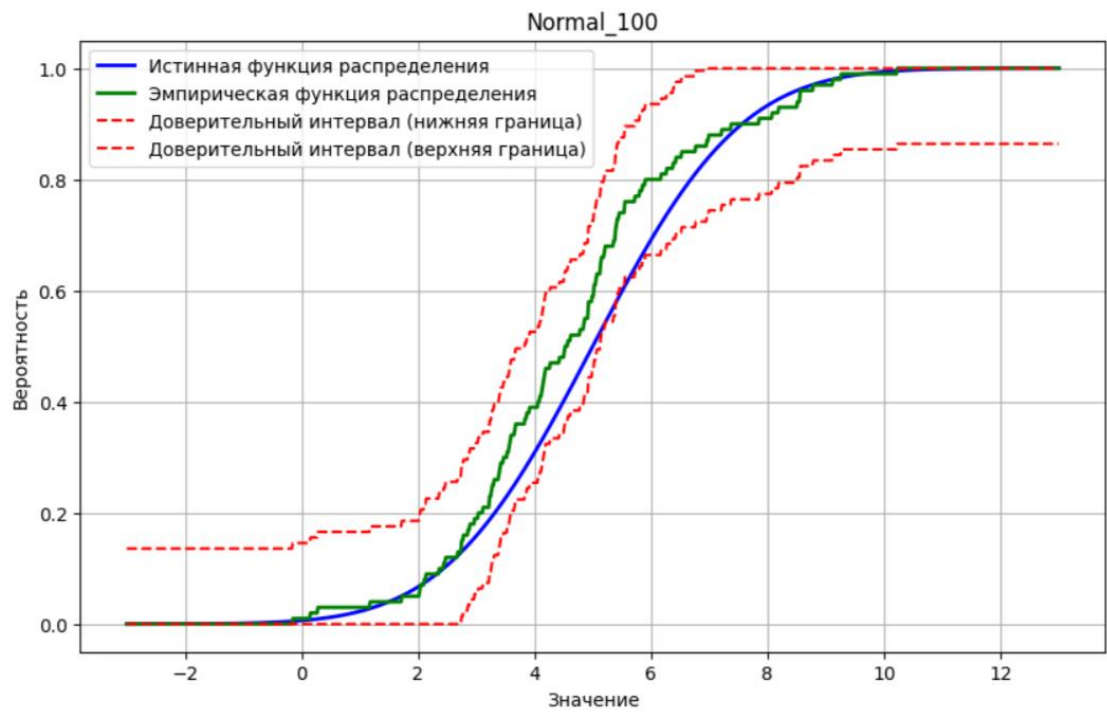
```
plt.legend()  
plt.grid(True)  
plt.show()
```

Вывод результатов:









Код для построения графиков распределений с эмпирической функцией `scipy.stats`:

```
import scipy.stats as stats
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import ecdf, norm

np.random.seed(4)

uniform_params = {'loc': 5, 'scale': 10}
bernoulli_params = {'p': 0.7}
binomial_params = {'n': 20, 'p': 0.4}
normal_params = {'loc': 5, 'scale': 2}

samples = {
    'Uniform_100': stats.uniform.rvs(size=100, **uniform_params),
    'Uniform_1000': stats.uniform.rvs(size=1000, **uniform_params),
    'Bernoulli_100': stats.bernoulli.rvs(size=100, **bernoulli_params),
    'Bernoulli_1000': stats.bernoulli.rvs(size=1000, **bernoulli_params),
    'Binomial_100': stats.binom.rvs(size=100, **binomial_params),
    'Binomial_1000': stats.binom.rvs(size=1000, **binomial_params),
    'Normal_100': stats.norm.rvs(size=100, **normal_params),
    'Normal_1000': stats.norm.rvs(size=1000, **normal_params)
}

def empiricheskaya_fnc_scipy(sample, x):

    empirich = ecdf(x)

    n = len(sample)
    alpha = 0.05

    stand_err = np.sqrt(empirich * (1 - empirich) / n)

    ci = norm.interval(1 - alpha, loc=empirich, scale=stand_err)

    L_x = np.maximum(0, ci[0])
    U_x = np.minimum(1, ci[1])

    return empirich, L_x, U_x

colors = ['blue', 'green', 'red']

for name, sample in samples.items():
    if 'Uniform' in name:
```

```

        x_range = np.linspace(uniform_params['loc'], uniform_params['loc']
+ uniform_params['scale'], 1000)
        true_cdf = stats.uniform.cdf(x_range, **uniform_params)
        plot_type = 'plot'
    elif 'Bernoulli' in name:
        x_range = [0, 1]
        true_cdf = [1 - bernoulli_params['p'], 1]
        plot_type = 'step'
    elif 'Binomial' in name:
        x_range = np.arange(0, binomial_params['n'] + 1)
        true_cdf = stats.binom.cdf(x_range, **binomial_params)
        plot_type = 'step'
    elif 'Normal' in name:
        x_range = np.linspace(normal_params['loc'] - 4 *
normal_params['scale'],
                                normal_params['loc'] + 4 *
normal_params['scale'], 1000)
        true_cdf = stats.norm.cdf(x_range, **normal_params)
        plot_type = 'plot'

    empirical_cdf = []
    lower_bounds = []
    upper_bounds = []
    for x in x_range:
        emp, L_x, U_x = empiricheskaya_fnc(sample, x)
        empirical_cdf.append(emp)
        lower_bounds.append(L_x)
        upper_bounds.append(U_x)

    plt.figure(figsize=(10, 6))

    if plot_type == 'plot':
        plt.plot(x_range, true_cdf, color=colors[0], label='Истинная
функция распределения', lw=2)
        plt.plot(x_range, empirical_cdf, color=colors[1],
label='Эмпирическая функция распределения', lw=2)
        plt.plot(x_range, lower_bounds, color=colors[2], linestyle='--',
label='Доверительный интервал (нижняя граница)')
        plt.plot(x_range, upper_bounds, color=colors[2], linestyle='--',
label='Доверительный интервал (верхняя граница)')
    elif plot_type == 'step':
        plt.step(x_range, true_cdf, where='post', color=colors[0],
label='Истинная функция распределения', lw=2)
        plt.step(x_range, empirical_cdf, where='post', color=colors[1],
label='Эмпирическая функция распределения', lw=2)
        plt.step(x_range, lower_bounds, where='post', color=colors[2],
linestyle='--', label='Доверительный интервал (нижняя граница)')
        plt.step(x_range, upper_bounds, where='post', color=colors[2],
linestyle='--', label='Доверительный интервал (верхняя граница)')

```

```
plt.title(f'{name}')
```

```
plt.xlabel('Значение')
```

```
plt.ylabel('Вероятность')
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```

Графики выводятся те же